Comprehensive Project Guide - Modern Buffer Overflow Lab

1. Project Summary

This project is a Docker-based penetration testing lab for teaching the full beginner workflow of binary exploitation:

```
Recon -> Fuzzing -> Crash Analysis -> Offset Discovery -> Exploitation
      -> Restricted Shell -> Post-Exploitation Basics -> Mitigation
```

The lab is intentionally controlled. It does not open a real unrestricted operating-system shell. Successful exploitation opens a restricted training shell inside the Docker victim so students can safely practice basic post-exploitation commands.

Use this project only in the provided local Docker lab environment.

2. Learning Objectives

By the end of the lab, students should understand:

1. How a vulnerable network service can be discovered.
2. How fuzzing finds a crash.
3. How a stack-based buffer overflow overwrites control data.
4. Why the return address matters.
5. How a ret2win exploit proves control-flow hijacking.
6. How NX changes the exploitation strategy.
7. Why stack canaries and PIE make exploitation harder.
8. How an information leak can bypass protections.
9. What a restricted shell represents in post-exploitation.
10. How defensive coding and compiler protections stop the attack.

3. Project Structure

```
college-pentest-lab/
|-- docker-compose.yml
|-- docker-compose.debug.yml
|-- README.md
|-- START_HERE.md
|-- attacker/
|   |-- Dockerfile
|   `-- scripts/
|       |-- 00_recon.py
|       |-- 01_fuzzer.py
|       |-- 02_find_offset.py
|       |-- 03_exploit_level1.py
|       |-- 04_exploit_level2.py
|       |-- 05_exploit_level3.py
|       `-- 06_verify_safe.py
|-- victim/
|   |-- Dockerfile
|   |-- vuln_server.c
|   |-- safe_server.c
|   `-- flag.txt
`-- docs/
    |-- debugging.md
```

```
    |-- COMPREHENSIVE_PROJECT_GUIDE.md
    `-- PRESENTATION_WALKTHROUGH.md
```

4. Docker Architecture

The project uses five containers on one internal Docker network:

```
attacker
  Runs Python exploit scripts and analysis tools.

victim-l1
  Vulnerable Level 1 service.
  Host port: 9999

victim-l2
  Vulnerable Level 2 service.
  Host port: 9998

victim-l3
  Vulnerable Level 3 service.
  Host port: 9997

safe-victim
  Defensive version of the service.
  Host port: 9996
```

Inside the Docker network, every target listens on port `9999`. From the host, ports are mapped differently so all services can run at the same time.

5. Containers and Responsibilities

attacker

The attacker container includes:

```
Python 3
pwntools
gdb
pwndbg
checksec
ROPgadget
ropper
nmap
netcat
curl
```

The target binaries are copied automatically to:

```
/targets
```

This lets students run:

```
checksec --file /targets/level1
nm -g /targets/level1
objdump -d /targets/level1
```

victim-l1, victim-l2, victim-l3

All three vulnerable services are built from:

```
victim/vuln_server.c
```

The difference between the levels is the compiler options and enabled training features.

safe-victim

The safe service is built from:

```
victim/safe_server.c
```

It keeps the same basic interface but fixes the unsafe behavior.

6. Important Commands

Start the lab:

```
docker compose up --build -d
```

Check status:

```
docker compose ps
```

Enter the attacker:

```
docker exec -it attacker bash
```

Run the full learning path:

```
python3 scripts/00_recon.py
python3 scripts/01_fuzzer.py
python3 scripts/02_find_offset.py
python3 scripts/03_exploit_level1.py
python3 scripts/04_exploit_level2.py
python3 scripts/05_exploit_level3.py
python3 scripts/06_verify_safe.py
```

Run interactive restricted shell mode:

```
python3 scripts/03_exploit_level1.py --interactive
python3 scripts/04_exploit_level2.py --interactive
python3 scripts/05_exploit_level3.py --interactive
```

Reset everything:

```
docker compose down -v
docker compose up --build -d
```

7. The Vulnerability

The vulnerable server accepts commands over TCP.

Important command:

```
TRUN <data>
```

The vulnerable code copies attacker-controlled data into a fixed stack buffer without enforcing a safe length:

```
char buffer[128];
memcpy(buffer, input, input_len);
```

If `input_len` is larger than the stack buffer, data continues past the end of the buffer and overwrites nearby stack values.

8. Stack Layout Concepts

No Canary Layout

Level 1 and Level 2 use this beginner layout:

```
buffer[128]       offset 0
saved RBP[8]      offset 128
return address    offset 136
```

So the exploit payload is:

```
"A" * 136 + address_of(win)
```

Canary Layout

Level 3 uses stack canaries. The compiler adds padding before the canary:

```
buffer[128]       offset 0
padding[8]        offset 128
canary[8]         offset 136
saved RBP[8]      offset 144
return address    offset 152
```

So the exploit payload is:

```
"A" * 136 + leaked_canary + "B" * 8 + leaked_win
```

9. Restricted Training Shell

When exploitation succeeds, the server calls `win()` and opens a restricted training shell.

Allowed commands:

```
help
whoami
id
pwd
ls
cat flag.txt
hostname
uname -a
exit
```

This shell is intentionally limited. It teaches the idea of post-exploitation without exposing a real unrestricted `/bin/sh`.

10. Script-by-Script Explanation

00_recon.py

Purpose:

```
Identify services and compare binary protections.
```

What it does:

1. Connects to each service.
2. Reads the banner.
3. Runs `checksec` against the copied binaries in `/targets`.
4. Prints a protection summary.

Expected lesson:

```
Different protections require different exploitation strategies.
```

01_fuzzer.py

Purpose:

```
Find an input size that crashes the vulnerable service.
```

What it does:

1. Sends `TRUN` inputs of increasing length.
2. Watches whether the service responds.
3. Reports when the child process crashes.

Expected result:

```
Crash around 150 bytes.
```

Expected lesson:

```
The buffer is around 128 bytes, and extra bytes corrupt control data.
```

02_find_offset.py

Purpose:

```
Explain how to find the exact return-address offset.
```

What it does:

1. Sends a cyclic pattern.
2. Sends a marker at the expected offset.
3. Explains how to confirm the overwrite using Docker logs.

Important values:

```
Level 1 and Level 2 return-address offset: 136
Level 3 return-address offset with canary: 152
```

03_exploit_level1.py

Purpose:

```
Exploit Level 1 with ret2win.
```

Protections:

```
No canary
No PIE
Executable stack
```

Exploit idea:

```
Overwrite the saved return address with the address of win().
```

Payload:

```
"A" * 136 + p64(win)
```

Result:

```
win() runs, prints the flag, and opens the restricted training shell.
```

Main lesson:

```
Control over the return address means control over program execution.
```

04_exploit_level2.py

Purpose:

```
Show how NX changes the exploitation strategy.
```

Protections:

```
NX enabled
No canary
No PIE
```

NX means the stack is not executable. Instead of trying to run shellcode on the stack, the exploit reuses code already inside the binary.

Payload:

```
"A" * 136 + p64(win)
```

Result:

```
win() runs and opens the restricted training shell.
```

Main lesson:

```
When stack execution is blocked, attackers reuse existing code.
```

05_exploit_level3.py

Purpose:

```
Show why leaks matter when canary and PIE are enabled.
```

Protections:

```
Stack canary
NX
```

```
PIE
Full RELRO
```

Level 3 includes an intentional training leak through `INFO`.

Leak command:

```
INFO %1$p.%2$p
```

Leaked values:

```
%1$p -> stack canary
%2$p -> win() address after PIE relocation
```

Payload:

```
"A" * 136 + leaked_canary + "B" * 8 + leaked_win
```

Result:

```
The canary is preserved, PIE is bypassed using the leaked address, and win()
opens the restricted training shell.
```

Main lesson:

```
Modern exploitation often requires an information leak before control-flow
hijacking can succeed.
```

06_verify_safe.py

Purpose:

```
Prove that the defensive server blocks the attack path.
```

What it tests:

1. Normal input works.
2. Oversized input is rejected.
3. Format-string payload is printed as text.
4. The server remains stable.
5. Defense notes can be displayed.

Main lesson:

```
Input validation, bounded copying, safe format strings, and compiler hardening
stop the attack chain.
```

11. Level Comparison

Level	Canary	NX	PIE	Main Idea	Shell Result
1	No	No	No	Basic ret2win	Restricted shell
2	No	Yes	No	NX bypass with existing code	Restricted shell
3	Yes	Yes	Yes	Leak canary and PIE first	Restricted shell

Level	Canary	NX	PIE	Main Idea	Shell Result
Safe	Yes	Yes	Yes	Defensive implementation	Attack blocked

12. Safe Server Design

The safe server fixes the vulnerable behavior.

Input Length Validation

It rejects oversized input before copying:

```
if (input_len >= SAFE_BUFFER_SIZE) {
    send(client_fd, reject, strlen(reject), 0);
    return;
}
```

Bounded Copy

It copies only within the buffer:

```
strncpy(buffer, input, SAFE_BUFFER_SIZE - 1);
buffer[SAFE_BUFFER_SIZE - 1] = '\0';
```

Safe Format Strings

It treats user input as data:

```
snprintf(response, sizeof(response), "%s\n", input);
```

Compiler Protections

The safe server is compiled with:

```
Stack canary
NX
PIE
Full RELRO
FORTIFY_SOURCE
```

13. Suggested Student Tasks

1. Start the lab and confirm all containers are healthy.
2. Identify all services with `00_recon.py`.
3. Explain the difference between Level 1, Level 2, and Level 3.
4. Use fuzzing to find the crash threshold.
5. Explain why the offset is 136 in Level 1 and Level 2.
6. Exploit Level 1 and read the flag.
7. Explain why Level 2 does not use stack shellcode.
8. Exploit Level 2 and open the restricted shell.
9. Leak the canary and PIE address in Level 3.
10. Exploit Level 3 while preserving the canary.
11. Run the safe verification script.
12. Write a short penetration test report.

14. Report Template

```
Title:
Stack-Based Buffer Overflow in Vulnerable TCP Service

Affected Component:
TRUN command in vulnerable server.

Vulnerability:
Unbounded copy into a fixed-size stack buffer.

Impact:
An attacker can overwrite the saved return address and redirect execution.
In the lab, this opens a restricted training shell and reads the flag.

Proof of Concept:
Level 1 payload:
"A" * 136 + p64(win)

Level 2 payload:
"A" * 136 + p64(win)

Level 3 payload:
"A" * 136 + leaked_canary + "B" * 8 + leaked_win

Root Cause:
Attacker-controlled input is copied into a stack buffer without enforcing a
safe length.

Mitigation:
Validate input length before copying, use bounded copy logic, avoid
user-controlled format strings, and compile with hardening features.

Severity:
High in an unsafe native service.
```

15. Troubleshooting

Containers are not healthy

```
docker compose ps
docker compose logs victim-l1
docker compose logs safe-victim
```

Attacker cannot find binaries

Check:

```
docker exec -it attacker bash
ls -la /targets
```

If needed, rebuild:

```
docker compose down -v
docker compose up --build -d
```

Level 3 exploit fails

Rebuild and rerun because the PIE address and canary change:

```
docker compose restart victim-l3
docker exec -it attacker bash
python3 scripts/05_exploit_level3.py
```

Safe server blocks attacks

That is expected. The safe server should reject oversized input and treat format-string payloads as normal text.

16. Final Takeaway

This lab teaches a full beginner penetration testing story:

```
Find a service.
Understand its protections.
Crash it.
Find the overwrite offset.
Exploit control flow.
Open a controlled training shell.
Compare against secure code.
Explain the mitigation.
```

The goal is not to create a dangerous tool. The goal is to make students understand how memory corruption becomes impact, and how secure engineering prevents it.